

TABLA DE CONTENIDO

NIVEL FUNDAMENTAL	1
Módulo 6. Librería Estándar y E/S	1
Pathlib y manejo de archivos	1
CSV, JSON y YAML	2
'datetime' y Zona Horarias	5
Logging y Configuración	6
Módulo subprocess y Automatización	8
Preparación del Laboratorio	10

NIVEL FUNDAMENTAL

MÓDULO 6. LIBRERÍA ESTÁNDAR Y E/S

PATHLIB Y MANEJO DE ARCHIVOS

pathlib. Permite trabajar con *rutas* y *archivos* mediante objetos, en lugar de concatenar cadenas manualmente. El operador `/` combina partes de la ruta de forma segura, y permite *encadenar varias partes*. “Lo que hace es representar la ubicación del archivo”.

```
from pathlib import Path
ruta = Path("datos") / "ventas.csv"
# El objeto ruta apunta a datos/ventas.csv
```

Ruta del Proyecto. Siguiendo el siguiente ejemplo:

```
ruta_actual = Path(__file__).parent
```

Si el archivo está en **src/main.py**, entonces `Path(__file__).parent` representa **src/**. Y para acceder a la raíz del proyecto:

```
raiz = Path(__file__).parent.parent
```

Comprobar Archivos y Carpetas. Siguiendo el ejemplo:

<code>ruta = Path("datos/ventas.csv")</code>	<code>exists()</code>	Verifica si existe.
<code>print(ruta.exists())</code>	<code>is_file()</code>	Verifica si es un archivo.
<code>print(ruta.is_file())</code>	<code>is_dir()</code>	Verifica si es un directorio.
<code>print(ruta.is_dir())</code>		

Crear Directorios.

```
directorio = Path("salidas")
directorio.mkdir(exist_ok=True)
```

Directorio

```
directorio.mkdir(parents=True, exist_ok=True)
```

Directorio padre.

Leer Archivos.

```
ruta = Path("datos.txt")
contenido = ruta.read_text(encoding="utf-8")
print(contenido)
```

Lee el archivo de la ruta.

```
with ruta.open(encoding="utf-8") as archivo:
    contenido = archivo.read()
```

Utiliza open mediante un context manager.

Escribir Archivos.

```
ruta.write_text("Contenido nuevo",
encoding="utf-8")
```

Reemplaza el contenido existente.

```
with ruta.open("a", encoding="utf-8") as
archivo:
    archivo.write("\nNueva línea")
```

Agrega contenido al final.

Recorrer Archivos.

```
directorio = Path("datos")
```

Recorrer archivo completo.

```
for archivo in directorio.iterdir():
    print(archivo)
```

```
for archivo in directorio.glob("*.csv"):
    print(archivo)
```

Buscar archivos específicos.

Rutas Absolutas y Relativas. La ruta relativa depende de la carpeta desde la que se ejecuta el programa, mientras que la ruta absoluta contiene toda la ubicación.

```
Path("datos/ventas.csv")
```

Ruta relativa definida (según cada caso).

```
ruta_absoluta =
Path("datos/ventas.csv").resolve()
```

Para obtener una ruta absoluta.

Ventaja de Rutas frente a Cadenas.

```
ruta = Path("datos") / "ventas.csv"
```

Uso de ruta.

```
ruta = "datos/" + "ventas.csv"
o
ruta = "datos\\ventas.csv"
```

La ruta evita usar alguno de estos casos.

CSV, JSON Y YAML

CSV. Representa datos tabulares separados por delimitadores, por ejemplo:

```
producto,cantidad,precio
Teclado,2,50.0
Monitor,1,200.0
```

Cada fila del CSV se obtiene como un **diccionario**, y a su vez, como cadenas.

```
import csv
from pathlib import Path

ruta = Path("datos/ventas.csv")

with ruta.open(newline="", encoding="utf-8") as archivo:
    lector = csv.DictReader(archivo)

    for fila in lector:
        print(fila)
```

En caso de utilizar los datos contenidos del CSV, se deben **convertir**.

```
cantidad = int(fila["cantidad"])
precio = float(fila["precio"])
```

Y para **escribir** en el archivo CSV:

```
with ruta.open("w", newline="", encoding="utf-8") as archivo:
    campos = ["producto", "cantidad", "precio"]
    escritor = csv.DictWriter(archivo, fieldnames=campos)

    escritor.writeheader()
    escritor.writerow({
        "producto": "Teclado",
        "cantidad": 2,
        "precio": 50.0,
    })
```

JSON. Representa objetos, listas y valores estructurados. Ejemplo:

```
import json

datos = {
    "producto": "Teclado",
    "cantidad": 2,
}
```

Serializar a texto JSON:

```
texto = json.dumps(datos, ensure_ascii=False, indent=2)
```

Escribir JSON en un archivo:

```
with Path("salida.json").open("w", encoding="utf-8") as archivo:
    json.dump(datos, archivo, ensure_ascii=False, indent=2)
```

Leer JSON:

```
with Path("salida.json").open(encoding="utf-8") as archivo:
    datos = json.load(archivo)
```

YAML. Formato legible para configuración y datos estructurados. Python *no incluye* YAML en la biblioteca estándar; se utiliza normalmente `poetry add pyyaml`. Ejemplo:

```
producto: Teclado
cantidad: 2
```

Leer YAML:

```
import yaml

with Path("config.yaml").open(encoding="utf-8") as archivo:
    configuracion = yaml.safe_load(archivo)
```

Escribir YAML:

```
with Path("config.yaml").open("w", encoding="utf-8") as archivo:
    yaml.safe_dump(configuracion, archivo, allow_unicode=True)
```

Usa `safe_load` y `safe_dump` para evitar la construcción de objetos arbitrarios.

Comparación entre formatos.

Formato	Uso Habitual	Biblioteca
CSV	<i>Datos tabulares.</i>	<code>csv</code>
JSON	<i>Intercambio de datos.</i>	<code>json</code>
YAML	<i>Configuración legible</i>	<code>PyYAML</code>

Parseo y Serialización. *Parsear* es convertir texto o archivo a estructuras Python. *Serializar* es convertir estructuras Python a texto o archivo.

```
json.load(archivo)      # Parseo desde archivo
json.dump(datos, archivo) # Serialización hacia archivo
```

Errores frecuentes. Con respecto a los formatos:

CSV:

- Columnas faltantes.
- Valores no numéricos.
- Delimitador incorrecto.

JSON:

- `json.JSONDecodeError` si el contenido es inválido.

YAML:

- Errores de sintaxis o estructura.
- Riesgo de seguridad si se utilizan funciones de carga no seguras.

'DATETIME' Y ZONA HORARIAS

Módulos de datetime. Se deben importar primero a Python (a excepción de timezone) desde el paquete 'datetime'.

<code>date</code>	<i>Solo fecha.</i>
<code>time</code>	<i>Solo hora.</i>
<code>datetime</code>	<i>Fecha y hora.</i>
<code>timedelta</code>	<i>Diferencia entre fechas y horas.</i>
<code>timezone</code>	<i>Información de zona horaria.</i>

Fecha Actual. Función '`.today()`'. El resultado depende de la fecha del sistema.

```
hoy = date.today()
print(hoy)
```

Fecha y Hora Actuales. Utiliza UTC '`.utc`'. El objeto contiene información de zona horaria, donde

```
from datetime import datetime, timezone
ahora = datetime.now(timezone.utc)
print(ahora)
```

Zonas Horarias. Para utilizar una zona concreta, utiliza '`zoneinfo`', la cual forma parte de la biblioteca estándar de Python (a partir de la 3.9).

```
from zoneinfo import ZoneInfo
from datetime import datetime

hora_madrid = datetime.now(ZoneInfo("Europe/Madrid"))
hora_mexico = datetime.now(ZoneInfo("America/Mexico_City"))
```

Fechas Conscientes y Naïve. Una fecha y hora '*naïve*' no contiene información de zona. Mientras que una '*aware*' sí contiene zona. Para sistemas que trabajan con *distintas regiones*, es preferible usar fechas *aware*.

<code>fecha_naive = datetime.now()</code>	<i>Naïve.</i>
<code>fecha_aware = datetime.now(timezone.utc)</code>	<i>Aware.</i>

Conversión de Texto a Fecha. Usa el siguiente formato:

Ejemplo	Código	Significado
<code>fecha = datetime.strptime("2026-09-16 14:30", "%Y-%m-%d %H:%M",)</code>	<code>%Y</code>	<i>Año.</i>
	<code>%m</code>	<i>Mes.</i>
	<code>%d</code>	<i>Día.</i>
	<code>%H</code>	<i>Hora.</i>
	<code>%M</code>	<i>Minuto.</i>

Conversión de Fecha a Texto. Usando el formato anterior:

```
texto = fecha.strftime("%d/%m/%Y %H:%M")
16/09/2026 14:30 # Resultado de ejemplo
```

Diferencias con `timedelta`. Añade la diferencia de tiempo en el argumento. También se puede usar para calcular diferencias.

```
ahora = datetime.now(timezone.utc)    Uso de timedelta.
mañana = ahora + timedelta(days=1)

diferencia = mañana - ahora          Para cálculo de
print(diferencia)                    diferencias.
```

Convertir entre Zonas. La misma fecha y hora se representa según otra zona.

```
hora_utc = datetime.now(timezone.utc)
hora_local = hora_utc.astimezone(timezone.get_default_timezone())
```

Recomendaciones. Para *sistemas distribuidos*:

- ❖ Guardar fechas en **UTC**.
- ❖ Utilizar objetos **aware** para evitar comparaciones ambiguas, errores durante cambios de verano e interpretaciones distintas del mismo registro temporal.
- ❖ Convierte a la zona local solo al mostrar.
- ❖ Evita mezclar fechas naïve y aware.

LOGGING Y CONFIGURACIÓN

logging permite registrar eventos de una aplicación con distintos niveles de importancia. Requiere importar el paquete `'logging'`. A diferencia de `print`, los logs pueden:

- Filtrarse por nivel.
- Incluir fecha y hora.
- Escribirse en archivos.
- Enviarse a distintos destinos.
- Configurarse de forma centralizada.

Niveles de Logging. Según cual se utilice, será el punto de inicio y solo se mostrarán los mensajes de mayor nivel, los de menor no. De menor a mayor severidad:

Nivel	Uso
DEBUG	Información detallada para diagnóstico.
INFO	Flujo normal de la aplicación.
WARNING	Situación inesperada, pero no necesariamente fallida.
ERROR	Error que afecta una operación.
CRITICAL	Error grave que puede detener el sistema.

Configuración Básica. Ejemplo:

```
import logging      # Paquete

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
)

logger = logging.getLogger(__name__)

logger.info("Proceso iniciado")      # Mensaje de flujo
logger.warning("El archivo contiene una fila incompleta") # Aviso
logger.error("No se pudo procesar el archivo")      # Mensaje de error
```

Uso de `logger.exception`. Se utiliza dentro de un `except` para registrar el mensaje y el traceback de la excepción actual.

```
try:
    resultado = int("abc")
except ValueError:
    logger.exception("No se pudo convertir el valor")
```

Diferencias de `print` y `logging`. Comparando:

print	logging
▪ Muestra texto directamente. ▪ No tiene niveles. ▪ No incluye contexto automáticamente. ▪ Es <i>menos configurable</i>.	▪ Tiene <i>niveles</i>. ▪ Permite formato consistente. ▪ Puede escribir en <i>consola y archivos</i>. ▪ Incluye información <i>temporal</i>. ▪ Facilita investigar problemas.

Logger por módulo. Se recomienda crear un logger por cada módulo, cuyo nombre será el módulo actual.

Handlers. Define a *dónde* se envían los logs, sea consola o archivo. Cada handler puede tener su propio nivel y formato.

```
Consola_handler = logging.StreamHandler() # Consola

File_handler=logging.FileHandler("app.log", encoding="utf-8") # Archivo
```

Configuración con `dictConfig`. Para aplicaciones más organizadas.

```
from logging.config import dictConfig

dictConfig({
    "version": 1,
    "disable_existing_loggers": False,
    "formatters": {      # Definición de formatos
        "standard": {
            "format": "%(asctime)s [%(levelname)s] %(name)s: %(message)s",
```

```

    },
},
"handlers": {      # Handlers para definir archivo destino
    "console": {    # Consola
        "class": "logging.StreamHandler",
        "formatter": "standard",
        "level": "INFO",
    },
},
"root": {
    "handlers": ["console"],
    "level": "INFO",
},
})

```

Logging Estructurado. Puede incluir *campos adicionales*; el formato debe estar definido para mostrar los campos que se desean visualizar. En aplicaciones reales, también puede utilizarse JSON para facilitar el análisis automático de logs.

```

logger.info(
    "Venta procesada",
    extra={
        "producto": "Teclado",
        "cantidad": 2,
    },
)

```

MÓDULO SUBPROCESS Y AUTOMATIZACIÓN

El módulo `subprocess` permite ejecutar *comandos externos* desde Python. Puede utilizarse para *automatizar tareas* como:

- ❖ Ejecutar herramientas del sistema.
- ❖ Consultar comandos.
- ❖ Lanzar procesos.
- ❖ Integrar scripts existentes.

Ejecutar un Comando Sencillo. El comando se proporciona como una lista.

```

resultado = subprocess.run(
    ["python", "--version"],
    capture_output=True,
    text=True,
)

```

Donde:

- `"python"`: Ejecutable
- `"--version"`: Argumento

Leer la Salida. Usando `.stdout`.

```

print(resultado.stdout)  Lee la salida estándar.
print(resultado.stderr)  Ubicación de los errores. text=True
                        convierte la salida de bytes a cadenas.

```

Comprobar el Código de Retorno. Según la respuesta de `.returncode`, por lo general: `'0'`
 – Ejecución correcta, *Otro valor* – Ocurrió un error.

Comprobación `check=True`. Para que Python lance una excepción si el *comando falla*.

```
subprocess.run(
    ["python", "--version"],
    check=True,
)
```

Si el comando devuelve un código diferente de cero, se produce `subprocess.CalledProcessError`.

Capturar Salida y Errores. Usando `capture_output` y `text`.

```
resultado = subprocess.run(
    ["python", "--version"],
    capture_output=True, # Salidas stdout y stderr sean separadas
    text=True, # stdout y stderr se devueven como cadenas de txt
    check=True,
)
```

Evitar `shell=True`. Su uso puede permitir *inyección de comandos*. Es preferible una *lista* que separa el ejecutable de sus argumentos. Por ejemplo, si en el código siguiente `valor` proviene de un usuario o fuente externo.

Mala Práctica	Buena Práctica
<pre>subprocess.run(f"comando {valor}", shell=True,)</pre>	<pre>subprocess.run(["comando", valor], check=True,)</pre>

Ejecutar Comandos Específicos de Windows. Dentro de la lista de argumentos pueden usarse comandos de Windows, sin embargo, es preferible usar *APIs* de Python o `pathlib` para que el código sea portable.

```
subprocess.run(
    ["cmd", "/c", "echo", "Hola"],
    check=True,
)
```

Usos Adecuados. `subprocess` es útil cuando:

- ❖ No existe una API equivalente en Python.
- ❖ Debes ejecutar una herramienta externa.
- ❖ Necesitas integrar un programa del sistema.
- ❖ Automatizas comandos controlados.

PREPARACIÓN DEL LABORATORIO

➤ Flujo del programa:

```
Construir rutas con pathlib
↓
Leer el CSV
↓
Convertir cantidades y precios
↓
Parsear fechas
↓
Convertir fechas a una zona local
↓
Calcular métricas
↓
Exportar métricas a JSON
↓
Registrar eventos con logging
```

➤ Métricas por calcular:

- Número de ventas.
- Unidades vendidas.
- Ingreso total.
- Producto con mayor ingreso.
- Fecha de generación.
- Zona horaria utilizada.

➤ Archivos de salida:

- Resultado en: **salidas/metricas.json**
- Logs escritos en: **logs/ventas.log**

➤ Errores por manejar:

- CSV inexistente.
- CSV vacío.
- Columnas faltantes.
- Cantidad no numérica.
- Precio no numérico.
- Fecha con formato inválido.
- Error al escribir el JSON.

➤ Relación con etapas anteriores:

- leer_ventas: creado en src/ingesta.py.
- calcular_metricas: creado en src/metricas.py.
- exportar_metricas: creado en src/exportacion.py.
- Uso de pathlib, logging, CSV y JSON.